



McGILL UNIVERSITY

DEPARTMENT OF MATHEMATICS AND STATISTICS

Hierarchical copula dependence models

Cedric Phillips

Supervised by Dr. Valiquette and Prof. Nešlehová

MATH 470: Honours Research Project

April 21, 2026

Contents

1	Introduction	1
1.1	Copulas	1
2	Literature review	2
2.1	Overview of the d -variate Bernoulli tree	2
2.2	Fréchet-Hoeffding bounds and the CIA	3
2.3	Aggregation	4
2.4	Dependence structure	5
3	Further developments of the theory	7
3.1	The correlation structure of the tree	7
3.1.1	Hadamard factorization	8
3.1.2	Estimation	9
3.2	Simulating the tree	9
3.2.1	Tree topology	9
3.2.2	The simulation puzzle	10
3.2.3	Pseudometric algorithm	10
3.2.4	Hadamard factorization algorithm	13
3.3	Validation	15
3.3.1	By Wald test	15
3.3.2	By simulation	16
4	Discussion	18
4.1	Implications	18
4.2	Directions for future work	18
4.2.1	Developments on the greedy algorithm	18
4.2.2	Measuring partial recovery	18
4.2.3	Sorting the Θ matrix	19
4.2.4	A more ‘delicate’ algorithm	19
4.3	Applications	20
4.3.1	Financial data	20
4.3.2	Weather extremes	20
	References	21

1 Introduction

In this work, we explore further the structure of Bernoulli hierarchical aggregation trees as proposed in a 2026 paper by Valiquette, Nešlehová, and Genest [VGN26]. This report is designed for readers with at least an advanced undergraduate background in probability, though some ideas from statistics and algorithm design will no doubt be helpful.

Multivariate Bernoulli distributions arise in many contexts. They often are made up of binary indicator random variables, which can be used, among other purposes, to report on extreme or tail events. One might then be interested in the way in which the marginals of the Bernoulli random variable are dependent. However, the random vector is often intractable in higher dimensions. For this reason, a parsimonious aggregation tree structure can be introduced. In this report, we work with maximum-style aggregation, as proposed in [VGN26]. We review the theory behind maximum aggregation trees, make explicit the structure of dependence within the trees, and attempt to leverage these properties to design an algorithm which can learn the true structure of such a tree given only raw data.

1.1 Copulas

Definition 1 ([Nel06]). A d -dimensional copula $C : [0, 1]^d \rightarrow [0, 1]$ is a cumulative distribution function (CDF) with uniform marginals:

$$C(u_1, u_2, \dots, u_d) = \mathbb{P}[U_1 \leq u_1, U_2 \leq u_2, \dots, U_d \leq u_d].$$

The theory of copulas is a rich subject, mostly predicated on a theorem of Sklar linking CDFs to their marginals and copulas. Previous works on hierarchical risk aggregation models [AHM12; CG15] have used copulas to aggregate continuous variables into tree structures. However, they have worked with continuous random variables; for discrete random variables, there are range-restriction and identifiability issues that make much of the copula toolbox difficult to work with. Nonetheless, it will be useful to introduce another result:

Proposition 2 ([Nel06]). *The Fréchet-Hoeffding bounds for a copula $C(u_1, u_2, \dots, u_d)$ are*

$$\max \left\{ 0, 1 - d + \sum_{j=1}^d u_j \right\} \leq C(u_1, u_2, \dots, u_d) \leq \min\{u_1, u_2, \dots, u_n\}.$$

In particular, for a bivariate copula $C(u_1, u_2)$,

$$\max\{0, u_1 + u_2 - 1\} \leq C(u_1, u_2) \leq \min\{u_1, u_2\}.$$

2 Literature review

This project has mostly been based on a model proposed in Valiquette et al. [VGN26]. The basic idea, as outlined by Arbenz et al. [AHM12], is to construct a hierarchical tree model from a random vector by aggregating small groups of marginals based on some measure of dependence. The work of Côté and Genest [CG15] attempts to specifically aggregate marginals into a *binary* tree using sums and a conditional independence assumption. This assumption and the binary tree model were then adapted by [VGN26] to the case of Bernoulli random variables.

2.1 Overview of the d -variate Bernoulli tree

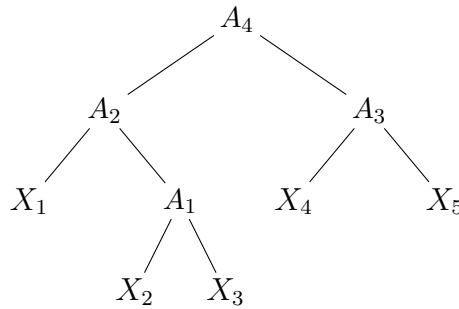
A d -variate Bernoulli distribution is a random vector $\mathbf{X} = (X_1, \dots, X_d)$ whose components take values in $\{0, 1\}$. Constructing a d -variate Bernoulli distribution typically requires specifying $2^d - 1$ parameters to fix the probabilities of all but one of the 2^d events made possible by various values of the X_i s. In practice, it might be instead desirable to fix the marginal distributions of $X_1, \dots, X_d$, giving (with an unusual convention) the values

$$p_i = \mathbb{P}[X_i = 0], \quad i \in \{1, \dots, d\}.$$

Then all that remains is to specify the ‘higher-level’ joint probabilities.

[VGN26] proposes propagating the dependence between the marginal variables $X_1, \dots, X_n$ according to a binary rooted tree structure, defining a bivariate Bernoulli random variable that combines at each interior node A_j of the tree. In particular, each A_j represents a random variable that uses a binary operation to combine its two children into one.

As an example, consider the following binary aggregation tree on $d = 5$ leaves.



Then, once the tree structure and binary operation have been properly specified, the joint distribution $(X_1, \dots, X_n)$ requires a specification of only $2d - 1$ parameters, which in higher dimensions is a significant improvement over the standard $2^d - 1$ required [VGN26].

2.2 Fréchet-Hoeffding bounds and the CIA

For each internal node A_i of the tree with left child $A_{i,1}$ and right child $A_{i,2}$, $i \in \{1, \dots, d-1\}$, the dependence parameter is defined to be

$$r_{A_i} = \mathbb{P}[A_{i,1} = 0, A_{i,2} = 0],$$

which characterizes the joint distribution of the two children at that node. Then $(A_{i,1}, A_{i,2})$ follows a bivariate Bernoulli distribution with parameters $p_{A_{i,1}}$, $p_{A_{i,2}}$, and r_{A_i} .

Definition 3. A random pair (X_1, X_2) with $X_1, X_2 \in \{0, 1\}$ follows a *bivariate Bernoulli distribution* with parameters $p_1 = \mathbb{P}[X_1 = 0]$, $p_2 = \mathbb{P}[X_2 = 0]$, and $r = \mathbb{P}[X_1 = 0, X_2 = 0]$, if its probability mass function (PMF) is given by

$$\begin{aligned} \mathbb{P}[X_1 = 0, X_2 = 0] &= r, \\ \mathbb{P}[X_1 = 0, X_2 = 1] &= p_1 - r, \\ \mathbb{P}[X_1 = 1, X_2 = 0] &= p_2 - r, \\ \mathbb{P}[X_1 = 1, X_2 = 1] &= 1 - p_1 - p_2 + r. \end{aligned}$$

Corollary 4 ([VGN26]). *For the bivariate Bernoulli distribution followed by $(A_{i,1}, A_{i,2})$ to be a valid, the Fréchet-Hoeffding bounds require that*

$$\max(0, p_{A_{i,1}} + p_{A_{i,2}} - 1) \leq r_{A_i} \leq \min(p_{A_{i,1}}, p_{A_{i,2}})$$

Proof. By Sklar's theorem [Nel06], the joint distribution of $(A_{i,1}, A_{i,2})$ can be written as

$$\mathbb{P}[A_{i,1} \leq x_1, A_{i,2} \leq x_2] = C(u_1(x_1), u_2(x_2))$$

for some copula C , where u_1, u_2 are the marginal CDFs. However, for discrete random variables, C is uniquely determined only on the grid $\text{Range}(u_1) \times \text{Range}(u_2) = \{0, p_{A_{i,1}}, 1\} \times \{0, p_{A_{i,2}}, 1\}$, so all that matters is the value $C(p_{A_{i,1}}, p_{A_{i,2}}) = r_{A_i}$. In particular, infinitely many distinct copulas yield the same bivariate Bernoulli distribution, as long as they agree at this point. Applying the Fréchet-Hoeffding bounds (cf. Proposition 2) at this point gives the result. $\square$

The constraints imposed by the tree and these bounds still leave an infinite family of valid distributions, which motivates the introduction of an additional assumption.

Definition 5 (Conditional Independence Assumption [VGN26]). A d -random binary rooted tree $\mathcal{T}$ satisfies the *conditional independence assumption* (CIA) (as originally seen in [AHM12], later simplified by [CG15]), if, for every $i \in \{1, \dots, d-1\}$, the leaf descendants $\mathbf{X}_{\mathcal{A}_i}$ of the internal node A_i and the remaining leaves $\mathbf{X}_{\mathcal{A}_i^c}$ are conditionally independent given A_i .

In plain language, the only information shared between the two ‘sides’ of the tree at any internal node passes through that node’s random variable. This leads to much more parsimonious models, and allows the joint distribution of $\mathbf{X}$ to be uniquely determined.

2.3 Aggregation

As mentioned above, each non-root internal node A_i , the two children $A_{i,1}$ and $A_{i,2}$ are combined into a single Bernoulli random variable. Eight different commutative binary operations are possible; however, in this work, we use only maximum aggregation $A_i = \max(A_{i,1}, A_{i,2})$. This defines the *multivariate Bernoulli maximum aggregation-tree model*. The maximum is a commutative operator, but also is nice in that it ‘preserves’ much of the structure of the tree. In particular, with this aggregation, an internal node $A_i = \max(A_{i,1}, A_{i,2})$ equals zero if and only if both of its children are zero, so the ‘activation’ of any leaf (especially relevant if the leaf nodes are indicator variables) percolates upward through the tree. Also, since $A_i = 0$ if and only if $A_{i,1} = A_{i,2} = 0$, we have

$$p_{A_i} = \mathbb{P}[A_i = 0] = \mathbb{P}[A_{i,1} = 0, A_{i,2} = 0] = r_{A_i}.$$

Under maximum aggregation, the PMF of $\mathbf{X}$ from [VGN26] takes a cleaner form.

Corollary 6. *From the PMF given by Theorem 1 of [VGN26], by substituting the maximum aggregation, for any $\mathbf{x} \in \{0, 1\}^d$,*

$$\mathbb{P}[\mathbf{X} = \mathbf{x}] = \left(\prod_{j=1}^{d-1} \frac{\mathbb{P}[A_{j,1} = \max(\mathbf{x}_{\mathcal{A}_{j,1}}), A_{j,2} = \max(\mathbf{x}_{\mathcal{A}_{j,2}})]}{\mathbb{P}[A_{j,1} = \max(\mathbf{x}_{\mathcal{A}_{j,1}})] \mathbb{P}[A_{j,2} = \max(\mathbf{x}_{\mathcal{A}_{j,2}})]} \right) \prod_{i=1}^d \mathbb{P}[X_i = x_i],$$

where $\mathcal{A}_{j,1}$ and $\mathcal{A}_{j,2}$ are the leaf-descendant index sets of the children of A_j .

Proof. Under maximum aggregation, $A_j = \max(A_{j,1}, A_{j,2})$, so for a leaf configuration $\mathbf{x} \in \{0, 1\}^d$, the value taken by any internal node $A_{j,k}$ is $\max(\mathbf{x}_{\mathcal{A}_{j,k}})$ (one can see this by induction as the max of maxes is the max over all leaf descendants). Substituting this value into the factor from Theorem 1 of [VGN26] at the node A_j replaces the events $\{A_{j,k} = a_{j,k}\}$ by $\{A_{j,k} = \max(\mathbf{x}_{\mathcal{A}_{j,k}})\}$, with the factor $\prod_{i=1}^d \mathbb{P}[X_i = x_i]$ falling in from Theorem 1 of [VGN26]. $\square$

The terms in the product thus reduce to probabilities involving r_{A_j} , $r_{A_{j,1}}$, and $r_{A_{j,2}}$, themselves dependence parameters from lower in the tree.

2.4 Dependence structure

For a d -variate Bernoulli random vector $\mathbf{X}$, the Pearson correlation between (X_i, X_j) is

$$\rho_{ij} = \text{Corr}(X_i, X_j) = \frac{r_{ij} - p_i p_j}{\sqrt{p_i(1-p_i)p_j(1-p_j)}},$$

see, e.g., [DDW13].

Proposition 7. *Suppose $\mathbf{X}$ follows a multivariate Bernoulli maximum aggregation model for a tree $\mathcal{T}$. For any distinct leaves X_i and X_j , let S denote their first (i.e. lowest) common ancestor in the tree, and let S_1 be the left child and S_2 the right child of S . Then*

$$\mathbb{P}[X_i = X_j = 1] = (1-p_i)(1-p_j) \frac{1-r_{S_1}-r_{S_2}+r_S}{(1-r_{S_1})(1-r_{S_2})}.$$

Using this with the correlation formula, one obtains

$$\rho_{ij} = \sqrt{\frac{(1-p_i)(1-p_j)}{p_i p_j}} \frac{r_S - r_{S_1} r_{S_2}}{(1-r_{S_1})(1-r_{S_2})} = \sqrt{\frac{(1-p_i)(1-p_j)r_{S_1}r_{S_2}}{p_i p_j (1-r_{S_1})(1-r_{S_2})}} \cdot \text{Corr}(S_1, S_2).$$

This result is from [VGN26]; the proof below was done as an exercise during the project.

Proof. Let S denote the first common ancestor of X_i and X_j , with X_i in the subtree below S_1 and X_j in the subtree below S_2 . By the CIA, given S_1 , X_i is conditionally independent of X_j (and similarly given S_2); moreover X_i is conditionally independent of S_2 given S_1 and vice versa. Conditioning on (S_1, S_2) ,

$$\mathbb{P}[X_i = 1, X_j = 1] = \sum_{a,b \in \{0,1\}} \mathbb{P}[X_i = 1 \mid S_1 = a] \mathbb{P}[X_j = 1 \mid S_2 = b] \mathbb{P}[S_1 = a, S_2 = b].$$

Under maximum aggregation, $S_1 = 0$ if and only if every leaf descendant of S_1 is 0; so $\mathbb{P}[X_i = 1 \mid S_1 = 0] = 0$. Applying the law of total probability to X_i ,

$$1-p_i = \mathbb{P}[X_i = 1] = \mathbb{P}[X_i = 1 \mid S_1 = 0] r_{S_1} + \mathbb{P}[X_i = 1 \mid S_1 = 1](1-r_{S_1}),$$

so $\mathbb{P}[X_i = 1 \mid S_1 = 1] = (1-p_i)/(1-r_{S_1})$. Similarly $\mathbb{P}[X_j = 1 \mid S_2 = 1] = (1-p_j)/(1-r_{S_2})$. Then

$$\mathbb{P}[S_1 = 1, S_2 = 1] = 1 - \mathbb{P}[S_1 = 0 \text{ or } S_2 = 0] = 1 - r_{S_1} - r_{S_2} + r_S$$

by inclusion-exclusion. Therefore

$$\mathbb{P}[X_i = 1, X_j = 1] = \frac{1-p_i}{1-r_{S_1}} \cdot \frac{1-p_j}{1-r_{S_2}} \cdot (1-r_{S_1}-r_{S_2}+r_S) = (1-p_i)(1-p_j) \frac{1-r_{S_1}-r_{S_2}+r_S}{(1-r_{S_1})(1-r_{S_2})}.$$

For the correlation, recall that $\mathbb{P}[X_i = 1] = 1 - p_i$ and $\text{Var}(X_i) = p_i(1 - p_i)$. Then

$$\begin{aligned} \rho_{ij} &= \frac{\mathbb{P}[X_i = 1, X_j = 1] - (1 - p_i)(1 - p_j)}{\sqrt{p_i(1 - p_i)p_j(1 - p_j)}} \\ &= \sqrt{\frac{(1 - p_i)(1 - p_j)}{p_i p_j}} \cdot \frac{(1 - r_{S_1} - r_{S_2} + r_S) - (1 - r_{S_1})(1 - r_{S_2})}{(1 - r_{S_1})(1 - r_{S_2})}. \end{aligned}$$

Expanding $(1 - r_{S_1})(1 - r_{S_2})$ simplifies the numerator to $r_S - r_{S_1}r_{S_2}$. Factoring using $\text{Corr}(S_1, S_2) = (r_S - r_{S_1}r_{S_2})/\sqrt{r_{S_1}(1 - r_{S_1})r_{S_2}(1 - r_{S_2})}$ yields the stated identity. $\square$

Consequently, the correlation between any two leaves X_i and X_j is proportional to the correlation between the children of their first common ancestor, so the sign of ρ_{ij} is determined by the dependence at that ancestor. In particular, all pairs of leaves with the same first common ancestor have the same sign of correlation.

In the symmetric case where $p_i = 1/2$ for all i , the expression simplifies further to

$$\rho_{ij} = \sqrt{\frac{r_{S_1} r_{S_2}}{(1 - r_{S_1})(1 - r_{S_2})}} \cdot \text{Corr}(S_1, S_2),$$

so all pairs sharing the same common ancestor share the same correlation value.

3 Further developments of the theory

The first portion of the time spent on this project was dedicated to a literature review aimed at increasing familiarity with the multivariate Bernoulli maximum aggregation-tree model. In the second phase of the project, we were concerned with ways to learn the tree.

Put simply: all these ideas about what is possible with such a tree $\mathcal{T}$ are well and good, but in reality, we often don't know the structure of $\mathcal{T}$. Suppose $\mathbf{X}$ is a random vector for which the Bernoulli maximum aggregation-tree model is appropriate, for some specific tree $\mathcal{T}$. How can we estimate or 'learn' $\mathcal{T}$ from the data of $\mathbf{X}$?

3.1 The correlation structure of the tree

Recall from the previous section that for a maximum aggregation tree, the correlation between any pair of leaves (X_i, X_j) with first common ancestor S (having children S_1, S_2) is given by

$$\rho_{ij} = \underbrace{\sqrt{\frac{(1-p_i)(1-p_j)r_{S_1}r_{S_2}}{p_i p_j (1-r_{S_1})(1-r_{S_2})}}}_{\text{marginal prefactor} > 0} \cdot \underbrace{\text{Corr}(S_1, S_2)}_{\text{sign depends on } r_S}.$$

Proposition 8 ([Val26]). *For any two distinct leaves X_i, X_j in the tree with first common ancestor S having children S_1, S_2 , we have*

$$|\rho_{ij}| \leq |\text{Corr}(S_1, S_2)|,$$

with equality if and only if the marginal prefactor equals 1.

Proof. Assuming that the marginals are not degenerate, the prefactor

$$\sqrt{\frac{(1-p_i)(1-p_j)r_{S_1}r_{S_2}}{p_i p_j (1-r_{S_1})(1-r_{S_2})}}$$

is strictly positive, and the sign of ρ_{ij} is determined by $\text{Corr}(S_1, S_2)$, which depends on the relationship between r_S and $r_{S_1} r_{S_2}$. By the Fréchet-Hoeffding bounds, the dependence parameter r_S satisfies

$$\max(0, r_{S_1} + r_{S_2} - 1) \leq r_S \leq \min(r_{S_1}, r_{S_2}),$$

so if $r_S = r_{S_1} r_{S_2}$, the children S_1 and S_2 are independent, and all leaf descendants of S_1 are independent of all leaf descendants of S_2 . Moreover, the Fréchet-Hoeffding bounds applied to the pairs (X_i, S_1) and (X_j, S_2) yield $r_{S_1} \leq p_i$ and $r_{S_2} \leq p_j$ (since $X_i = 0$ whenever $S_1 = 0$, and similarly for X_j). Then

$$\frac{(1-p_i)r_{S_1}}{p_i(1-r_{S_1})} \leq \frac{1-p_i}{1-r_{S_1}} \cdot \frac{p_i}{p_i} \leq 1,$$

since $r_{S_1} \leq p_i$, so $1 - r_{S_1} \geq 1 - p_i$. The same argument applies to the other factor. Multiplying,

$$\sqrt{\frac{(1-p_i)(1-p_j)r_{S_1}r_{S_2}}{p_i p_j (1-r_{S_1})(1-r_{S_2})}} \leq 1,$$

giving the desired inequality. $\square$

Therefore leaves which are closest in the tree are those which are freest to approach this bound. In particular, for siblings X_i, X_j , their first common ancestor is their parent, so they themselves are S_1, S_2 . This motivates us to use the correlation structure to learn the tree.

3.1.1 Hadamard factorization

Definition 9. (Hadamard product). For two matrices X and Y of the same dimension $m \times n$, their Hadamard product $X \circ Y$ has entries given by

$$(X \circ Y)_{ij} = (X_{ij}) \cdot (Y_{ij}).$$

Definition 10. The correlation matrix ρ of $\mathbf{X}$ admits a convenient decomposition as a Hadamard product of two matrices. Define

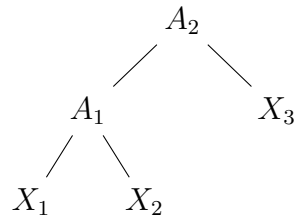
$$\theta_{ij} = \frac{r_{ij} - p_i p_j}{(1-p_i)(1-p_j)}.$$

Then $\rho_{ij} = \sqrt{\frac{(1-p_i)(1-p_j)}{p_i p_j}} \cdot \theta_{ij}$, so the full correlation matrix ρ can be written as

$$\rho = M \circ \Theta,$$

where $\circ$ denotes the Hadamard product, M is a matrix depending only on the marginals whose (i, j) -entry is $\sqrt{(1-p_i)(1-p_j)/(p_i p_j)}$, and Θ is a matrix whose (i, j) -entry is θ_{ij} .

Example 11. Consider a $d = 3$ tree with A_1 aggregating X_1, X_2 and A_2 aggregating A_1, X_3 :



The correlation matrix then decomposes as

$$\rho = \begin{pmatrix} 1 & \sqrt{\frac{(1-p_1)(1-p_2)}{p_1 p_2}} & \sqrt{\frac{(1-p_1)(1-p_3)}{p_1 p_3}} \\ \cdot & 1 & \sqrt{\frac{(1-p_2)(1-p_3)}{p_2 p_3}} \\ \cdot & \cdot & 1 \end{pmatrix} \circ \begin{pmatrix} 1 & \frac{r_1 - p_1 p_2}{(1-p_1)(1-p_2)} & \frac{r_2 - r_1 p_3}{(1-r_1)(1-p_3)} \\ \cdot & 1 & \frac{r_2 - r_1 p_3}{(1-r_1)(1-p_3)} \\ \cdot & \cdot & 1 \end{pmatrix},$$

where $r_1 = r_{A_1}$ and $r_2 = r_{A_2}$. Observe that $\theta_{13} = \theta_{23}$ in the second matrix, reflecting the fact that both pairs (X_1, X_3) and (X_2, X_3) share the same first common ancestor A_2 .

The idea is then to look for a block structure in the Θ matrix that we can exploit in order to learn the tree from data.

3.1.2 Estimation

The parameters can be estimated straightforwardly from an i.i.d. Bernoulli sample $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}$. The maximum likelihood estimators are

$$\hat{p}_i = \frac{1}{n} \sum_{k=1}^n \mathbf{1}_{\{X_i^{(k)}=0\}}, \quad \hat{r}_{ij} = \frac{1}{n} \sum_{k=1}^n \mathbf{1}_{\{X_i^{(k)}=X_j^{(k)}=0\}}.$$

Since the sample is i.i.d., $n\hat{p}_i \sim \text{Bin}(n, p_i)$ and $n\hat{r}_{ij} \sim \text{Bin}(n, r_{ij})$, so it is easy to check that these estimators are unbiased:

$$\mathbb{E}[\hat{p}_i] = \frac{1}{n} \sum_{k=1}^n \mathbb{P}[X_i^{(k)} = 0] = p_i, \quad \mathbb{E}[\hat{r}_{ij}] = \frac{1}{n} \sum_{k=1}^n \mathbb{P}[X_i^{(k)} = X_j^{(k)} = 0] = r_{ij}.$$

Their covariance structure can be computed directly. Letting $Y_{ik} = \mathbf{1}_{\{X_i^{(k)}=0\}}$, we have

$$\begin{aligned} \text{Cov}(\hat{p}_i, \hat{p}_j) &= \text{Cov}\left(\frac{1}{n} \sum_k Y_{ik}, \frac{1}{n} \sum_k Y_{jk}\right) = \frac{1}{n^2} \sum_{k=1}^n \text{Cov}(Y_{ik}, Y_{jk}) \\ &= \frac{1}{n} (\mathbb{E}[Y_{ik}Y_{jk}] - \mathbb{E}[Y_{ik}] \mathbb{E}[Y_{jk}]) = \frac{r_{ij} - p_i p_j}{n}, \end{aligned}$$

which gives $\text{Var}(\hat{p}_i) = p_i(1 - p_i)/n$ as the special case $i = j$.

3.2 Simulating the tree

Before applying any tree-learning algorithm to real data, it is useful to apply possible algorithms to data for which we know the true tree. This motivates us to simulate data from a known aggregation tree to test whether our algorithms can recover the original tree structure.

3.2.1 Tree topology

It is important to distinguish between *labeled* and *unlabeled* binary trees. A labeled tree has specific variables assigned to its leaves, while an unlabeled tree describes only the branching pattern. For the purpose of tree recovery, we consider two trees to be the ‘same’ if they have the same unlabeled topology.

Example 12. For example, the following two labeled trees on 8 leaves have the same unlabeled topology (a perfectly balanced tree):



The number of distinct unlabeled binary trees on d leaves is given by the Wedderburn–Etherington numbers [Eth37]. The first several values are:

d	2	3	4	5	6	7	8	9	10	...	20
# trees	1	1	2	3	6	11	23	46	98	...	293547

Our tree enumeration algorithm generates all unlabeled binary trees on d leaves by recursion. A single leaf is represented as the base case. For $d \geq 2$, a tree on d leaves is formed by choosing a left subtree on k leaves and a right subtree on $d - k$ leaves, for $k = 1, \dots, \lfloor d/2 \rfloor$. When $k < d - k$, the left and right subtrees have different sizes, so every pair (L, R) from the respective lists yields a distinct unlabeled tree. When $k = d - k$ (i.e. d is even and $k = d/2$), the left and right subtrees are drawn from the same list; to avoid counting the tree (L, R) and the tree (R, L) as distinct, we restrict to pairs where L appears no later than R in the order (i.e. if L is the i -th tree in the list, we only pair it with the j -th tree for $j \geq i$). The output was verified against the Wedderburn–Etherington sequence up to $d = 50$.

3.2.2 The simulation puzzle

Given the tree enumeration, we can set up the following ‘experiment’.

Algorithm 1 Generation of simulation puzzle

- 1: Generate all unlabeled binary trees on d leaves.
 - 2: Select a secret tree $\mathcal{T}$ uniformly at random from this collection.
 - 3: Assign marginal parameters $p_1, \dots, p_d$ and dependence parameters $r_{A_1}, \dots, r_{A_{d-1}}$ satisfying the Fréchet–Hoeffding bounds at each internal node.
 - 4: Simulate n i.i.d. observations from the resulting maximum aggregation-tree model, following the top-down conditional sampling scheme from [VGN26].
 - 5: Randomly permute the columns (leaves) of the resulting data matrix.
-

This algorithm is not particularly efficient in higher dimensions given that it generates all of the possible d –leaf trees each time it is run, which, given the growth of the Wedderburn–Etherington numbers, can become a problem for $d \geq 50$. Nonetheless, the ‘puzzle’ is then: given only the shuffled data matrix, can we recover the original tree topology?

3.2.3 Pseudometric algorithm

The first method, proposed in a note of Valiquette [Val26], is based on the pseudometric

$$D(X, Y) = \sqrt{1 - \text{Corr}(X, Y)^2}.$$

Proposition 13. $D(X, Y)$ is a pseudometric.

Proof. For any random variable X with positive variance, $\text{Corr}(X, X) = 1$, so $D(X, X) = \sqrt{1 - 1} = 0$. Furthermore, $D(X, Y) = D(Y, X)$ is immediate from the symmetry of Pearson correlation. The triangle inequality $D(X, Z) \leq D(X, Y) + D(Y, Z)$ follows from the positive semidefiniteness of the correlation matrix of (X, Y, Z) ; [Val26] gave a proof in the style of one by [CG15]. Hence D is a pseudometric. $\square$

However, it is not a metric; $D(X, Y) = 0$ only implies $|\text{Corr}(X, Y)| = 1$ (i.e. X and Y are perfectly correlated, and not necessarily $X = Y$).

Since minimizing D is equivalent to finding the most strongly correlated pair (in absolute value), we can propose the following algorithm.

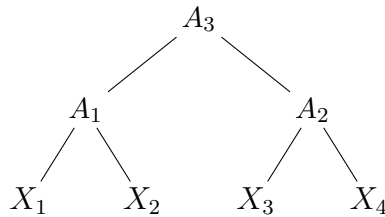
Algorithm 2 Pseudometric tree-recovery algorithm

- 1: Compute the empirical correlation matrix from the data.
 - 2: **while** more than one variable remains **do**
 - 3: Select the pair (X_i, X_j) that minimizes $D(X_i, X_j)$.
 - 4: Replace both columns X_i and X_j with $A = \max(X_i, X_j)$.
 - 5: Recompute the empirical correlation matrix from the updated dataset.
 - 6: **end while**
-

The motivation is from the inequality $|\rho_{ij}| \leq |\text{Corr}(S_1, S_2)|$: the most correlated pair should correspond to siblings in the tree (or at least to nodes whose first common ancestor is deepest).

Yet there is an important problem with this approach. At each merge step, the max-aggregation changes the marginal distribution of the aggregated variable: if $\mathbb{P}[X_i = 0] = p_i$ and $\mathbb{P}[X_j = 0] = p_j$, then $\mathbb{P}[A = 0] = r_A \leq \min(p_i, p_j)$, which can be smaller. This affects the marginal prefactor $\sqrt{(1 - p_A)(1 - p_k)/(p_A \cdot p_k)}$ in the correlation formula, inflating $|\text{Corr}(A, X_k)|$ relative to the ‘true’ tree-structural dependence. As a result, after the first merge, the aggregated node A can appear artificially close to the remaining leaves, causing the algorithm to merge A with the next leaf instead of merging other true siblings elsewhere in the tree, producing a *cascade* structure regardless of the true topology.

Example 14. Consider the balanced $d = 4$ tree with symmetric marginals $p_i = 1/2$:



Since $p_i = 1/2$ for all leaves, the marginal prefactors $\sqrt{(1 - p_i)(1 - p_j)/(p_i p_j)}$ all equal 1, so

every pairwise leaf correlation reduces to the corresponding θ value:

$$\begin{aligned}\text{Corr}(X_1, X_2) &= \theta_{A_1} = 4r_1 - 1, \\ \text{Corr}(X_3, X_4) &= \theta_{A_2} = 4r_2 - 1, \\ \text{Corr}(X_i, X_j) &= \theta_{A_3} = \frac{r_3 - r_1 r_2}{(1 - r_1)(1 - r_2)}, \quad i \in \{1, 2\}, j \in \{3, 4\}.\end{aligned}$$

Without loss of generality, assume $r_1 \geq r_2$ (so $\theta_{A_1} \geq \theta_{A_2}$), and suppose the dependence parameters satisfy a hierarchically ordered condition:

$$\theta_{A_1} \geq \theta_{A_2} > \theta_{A_3}, \quad \text{i.e.} \quad 4r_1 - 1 \geq 4r_2 - 1 > \frac{r_3 - r_1 r_2}{(1 - r_1)(1 - r_2)}.$$

This is the ‘correct’ ordering: the lower nodes A_1, A_2 contribute more dependence than the higher root A_3 , and the sibling pairs (X_1, X_2) and (X_3, X_4) are each more correlated than any cross-group pair.

Step 1. The algorithm computes all pairwise correlations and selects the pair with smallest D . Since $\theta_{A_1} \geq \theta_{A_2} > \theta_{A_3}$, this is (X_1, X_2) , and they are correctly merged into $A_1 = \max(X_1, X_2)$.

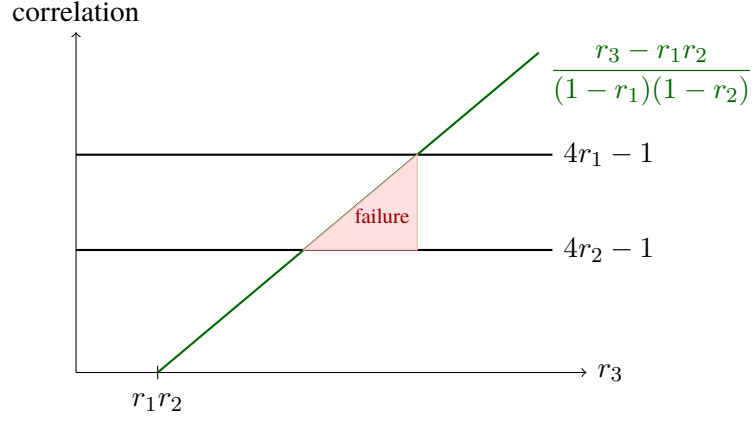
Step 2. After recomputing, the correlation $\text{Corr}(X_3, X_4) = 4r_2 - 1$ is unchanged. However, A_1 now has marginal $\mathbb{P}[A_1 = 0] = r_1 < 1/2$, while X_3 and X_4 still have marginal $1/2$. For any remaining leaf X_k with $k \in \{3, 4\}$, the recomputed correlation is

$$\text{Corr}(A_1, X_k) = \sqrt{\frac{(1 - r_1) \cdot \frac{1}{2}}{r_1 \cdot \frac{1}{2}}} \cdot \theta_{A_3} = \underbrace{\sqrt{\frac{1 - r_1}{r_1}}}_{>1} \cdot \theta_{A_3}$$

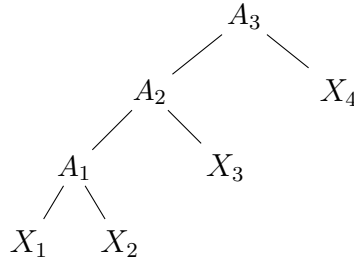
since $r_1 < 1/2$. The algorithm makes the wrong merge whenever this inflated correlation exceeds $\text{Corr}(X_3, X_4)$. As $\sqrt{r_1/(1 - r_1)} < 1$, the failure region

$$\theta_{A_2} \sqrt{\frac{r_1}{1 - r_1}} < \theta_{A_3} < \theta_{A_2}$$

is a nonempty subinterval of the values of θ_{A_3} permitted by the hierarchically ordered condition. In particular, for $r_1, r_2 \in (1/4, 1/2)$ with $r_1 \geq r_2$, there exist values of r_3 satisfying both the Fréchet-Hoeffding bounds and the hierarchically ordered condition for which the D -method fails, as illustrated in this diagram:



The result in these failure cases is then a cascade tree (seen below) instead of the correct balanced tree.



3.2.4 Hadamard factorization algorithm

The second method developed avoids the marginal-inflation problem by working directly with the θ_{ij} quantities from the Hadamard factorization.

Proposition 15. *Under the maximum aggregation-tree model with the CIA, for any two distinct leaves X_i and X_j with first common ancestor S having children S_1 and S_2 ,*

$$\theta_{ij} = \frac{r_{ij} - p_i p_j}{(1 - p_i)(1 - p_j)} = \frac{r_S - r_{S_1} r_{S_2}}{(1 - r_{S_1})(1 - r_{S_2})}.$$

In particular, θ_{ij} depends only on the dependence parameters at S , not on the leaves p_i, p_j . Consequently, all pairs of leaves with the same first common ancestor have the same θ_{ij} value.

This result is stated in [Val26]; the proof below was carried out as an exercise for the project.

Proof. Conditioning on (S_1, S_2) and using the CIA (so that $X_i \perp X_j$, $X_i \perp S_2$, and $X_j \perp S_1$),

$$r_{ij} = \sum_{a,b \in \{0,1\}} \mathbb{P}[X_i = 0 \mid S_1 = a] \mathbb{P}[X_j = 0 \mid S_2 = b] \mathbb{P}[S_1 = a, S_2 = b].$$

Under maximum aggregation, $S_1 = 0$ forces every leaf below S_1 to be 0, so $\mathbb{P}[X_i = 0 \mid S_1 = 0] = 1$. From the identity $\mathbb{P}[X_i = 0] = r_{S_1} + (1 - r_{S_1}) \mathbb{P}[X_i = 0 \mid S_1 = 1]$, define

$$\beta_i := \mathbb{P}[X_i = 0 \mid S_1 = 1] = \frac{p_i - r_{S_1}}{1 - r_{S_1}}, \quad \beta_j := \frac{p_j - r_{S_2}}{1 - r_{S_2}}.$$

Using $\mathbb{P}[S_1 = 0, S_2 = 0] = r_S$, $\mathbb{P}[S_1 = 0, S_2 = 1] = r_{S_1} - r_S$, $\mathbb{P}[S_1 = 1, S_2 = 0] = r_{S_2} - r_S$, and $\mathbb{P}[S_1 = 1, S_2 = 1] = 1 - r_{S_1} - r_{S_2} + r_S$, the sum becomes

$$r_{ij} = r_S + \beta_j(r_{S_1} - r_S) + \beta_i(r_{S_2} - r_S) + \beta_i\beta_j(1 - r_{S_1} - r_{S_2} + r_S).$$

Observe that $1 - p_i = (1 - r_{S_1})(1 - \beta_i)$ and $1 - p_j = (1 - r_{S_2})(1 - \beta_j)$, and similarly $p_i = r_{S_1} + (1 - r_{S_1})\beta_i$ and $p_j = r_{S_2} + (1 - r_{S_2})\beta_j$. Expanding the product $p_i p_j$, subtracting from r_{ij} , and canceling some β_i and β_j terms, one finds

$$r_{ij} - p_i p_j = (r_S - r_{S_1} r_{S_2})(1 - \beta_i)(1 - \beta_j).$$

Dividing by $(1 - p_i)(1 - p_j) = (1 - r_{S_1})(1 - \beta_i)(1 - r_{S_2})(1 - \beta_j)$ gives

$$\theta_{ij} = \frac{r_S - r_{S_1} r_{S_2}}{(1 - r_{S_1})(1 - r_{S_2})},$$

which depends only on the dependence parameters r_S, r_{S_1}, r_{S_2} at S . In particular, any two pairs of leaves sharing the same first common ancestor S yield the same value of θ_{ij} . $\square$

Since all pairs sharing the same first common ancestor therefore have the same θ_{ij} , the θ_{ij} values have a block structure that should directly mirror the tree hierarchy, without any ‘contamination’ like in the pseudometric algorithm.

Algorithm 3 Hadamard factorization tree-recovery (or θ) algorithm

- 1: Compute $\hat{\theta}_{ij}$ for all pairs from the empirical $\hat{p}_i$ and $\hat{r}_{ij}$ (cf. Section 3.1.2).
 - 2: **while** more than one variable remains **do**
 - 3: Select the pair (X_i, X_j) that maximizes $|\hat{\theta}_{ij}|$.
 - 4: Replace X_i and X_j with $A = \max(X_i, X_j)$ in the dataset.
 - 5: Recompute the $\hat{\theta}$ values between A and the remaining variables (using $\hat{p}_A$ and $\hat{r}_{A,k}$).
 - 6: **end while**
-

Definition 16 (Depth-ordered parametrization). A maximum aggregation-tree model satisfies the *depth-ordered parametrization* if, for any two pairs of leaves (X_i, X_j) and (X_k, X_l) whose first common ancestors S and S' are such that S is a proper descendant of S' (so S is strictly lower in $\mathcal{T}$), we have

$$|\theta_{ij}| > |\theta_{kl}|.$$

In plain language, pairs of leaves whose common ancestor is lower in the tree (further from the root) are strictly more dependent (in $|\theta|$) than pairs whose common ancestor is higher (closer to the root).

Proposition 17 (Correctness under depth-ordered parametrization). *Suppose the true tree $\mathcal{T}$ satisfies the depth-ordered parametrization. Then at each step of the Hadamard factorization algorithm, the pair of variables maximizing $|\theta_{ij}|$ in population (using the true parameters) are siblings in $\mathcal{T}$, and given the true parameters, the algorithm recovers $\mathcal{T}$.*

Proof. By induction on d . For the base case $d = 2$, there is nothing to prove. For the inductive step, consider a tree on $d \geq 3$ leaves satisfying this parametrization. The deepest possible first common ancestor of a pair of leaves is a node whose two children are both leaves; the pairs with such an ancestor are precisely siblings. By the depth-ordered parametrization, any such sibling pair has strictly larger $|\theta_{ij}|$ than any non-sibling pair, because the first common ancestor of a non-sibling pair is a proper ancestor of some sibling-parent node. Therefore the first pair selected by the algorithm is a true sibling pair, and the first merge is correct.

After merging the sibling pair (X_i, X_j) into $A = \max(X_i, X_j)$, the resulting collection of variables corresponds exactly to the tree obtained from $\mathcal{T}$ by collapsing the two leaves into their parent node A_{ij} , leaving a maximum aggregation tree on $d - 1$ leaves with the same dependence parameters at all remaining nodes. In particular, for any remaining leaf X_k , the first common ancestor of A and X_k in the reduced tree is the same node S' that was the first common ancestor of X_i and X_k in the original tree, with the same children S'_1, S'_2 . Hence

$$\theta_{A, X_k} = \frac{r_{S'} - r_{S'_1} r_{S'_2}}{(1 - r_{S'_1})(1 - r_{S'_2})} = \theta_{X_i, X_k},$$

so the surviving θ values are a subset of the original θ values, preserving their depth-ordering. Inductively, the remaining merges of are correct and the algorithm recovers $\mathcal{T}$. $\square$

3.3 Validation

3.3.1 By Wald test

Once a tree structure $\mathcal{T}$ has been learned, we can test whether the data are consistent with it. The key observation, as in Proposition 15, is that all pairs sharing the same first common ancestor must have the same θ_{ij} .

Formally, by the Delta Method applied to the transformation $(\hat{p}, \hat{r}) \mapsto \hat{\theta}$, we have

$$\sqrt{n}[\hat{\theta} - \theta] \xrightarrow{d} \mathcal{N}(\mathbf{0}, J_\theta \Sigma J_\theta^\top),$$

where $\theta = (\theta_{12}, \dots, \theta_{d-1, d})^\top$, Σ is the covariance matrix of $(\hat{p}, \hat{r})$, and J_θ is the Jacobian matrix with nonzero entries [Val26]

$$\frac{\partial \theta_{ij}}{\partial p_i} = \frac{r_{ij} - p_j}{(1 - p_i)^2(1 - p_j)}, \quad \frac{\partial \theta_{ij}}{\partial p_j} = \frac{r_{ij} - p_i}{(1 - p_i)(1 - p_j)^2}, \quad \frac{\partial \theta_{ij}}{\partial r_{ij}} = \frac{1}{(1 - p_i)(1 - p_j)}.$$

Corollary 18 ([Val26]). *Under the maximum aggregation-tree model with tree $\mathcal{T}$ and the CIA, the vector $\boldsymbol{\theta} = (\theta_{12}, \dots, \theta_{d-1,d})^\top$ satisfies*

$$R\boldsymbol{\theta} = \mathbf{0},$$

where R is the $\binom{d-1}{2} \times \binom{d}{2}$ restriction matrix encoding, for each internal node A_j , the equality of θ_{ij} across all pairs of leaves (X_i, X_j) sharing first common ancestor A_j .

Proof. Fix any internal node A_j with children $A_{j,1}$ and $A_{j,2}$, and let $(X_i, X_{i'})$ and (X_k, X_l) be two pairs of leaves whose first common ancestor is exactly A_j . By Proposition 15,

$$\theta_{i,i'} = \frac{r_{A_j} - r_{A_{j,1}} r_{A_{j,2}}}{(1 - r_{A_{j,1}})(1 - r_{A_{j,2}})} = \theta_{k,l}.$$

Hence all θ_{ij} values within the same ancestor group are equal. Each such equality $\theta_{i,i'} = \theta_{k,l}$ corresponds to the linear constraint $\theta_{i,i'} - \theta_{k,l} = 0$, corresponding to a row of R with $+1$ in the column for pair (i, i') and -1 in the column for pair (k, l) . Choosing one independent equality per ancestor group yields $\binom{d-1}{2}$ rows in total, one per degree of freedom in the Wald test. $\square$

The Wald test statistic is therefore

$$W = n(R\hat{\boldsymbol{\theta}})^\top \left[R \hat{J}_\theta \hat{\Sigma} \hat{J}_\theta^\top R^\top \right]^{-1} R\hat{\boldsymbol{\theta}} \xrightarrow{d} \chi^2_{\binom{d-1}{2}}$$

under H_0 [Val26]. One then rejects the model at level α if W exceeds the critical value $\chi^2_{\binom{d-1}{2}}$.

3.3.2 By simulation

To evaluate the Algorithm 3, we ran the simulation puzzle from section 3.2.2 in Python across a range of dimensions d , sample sizes n , and dependence strengths r_{scale} . At each internal node A_i with children having marginals $p_{A_{i,1}}$ and $p_{A_{i,2}}$, the dependence parameter is set to

$$r_{A_i} = r_{\min} + r_{\text{scale}} \cdot (r_{\max} - r_{\min}),$$

for the Fréchet–Hoeffding bounds $r_{\min} = \max(0, p_{A_{i,1}} + p_{A_{i,2}} - 1)$, $r_{\max} = \min(p_{A_{i,1}}, p_{A_{i,2}})$.

We also considered two types of marginals: symmetric, for which each leaf has $p_i = 0.5$, and heterogeneous, where each leaf has $p_i \sim \text{Uniform}[0.3, 0.7]$. For each configuration, we generated a number of random trees, simulated data, shuffled the leaves, and checked whether the algorithm recovered the correct unlabeled topology. We also subsequently developed a variant ‘oracle’ algorithm. The results for one seed are summarized in the table below.

d	n	r_{scale}	marginals	trials	standard recovery	oracle recovery
5	5000	0.7	symmetric	60	60/60	60/60
5	10000	0.7	symmetric	60	60/60	60/60
5	5000	0.7	heterogeneous	60	33/60	36/60
5	10000	0.7	heterogeneous	60	32/60	37/60
5	10000	0.3	symmetric	60	36/60	43/60
5	10000	0.5	symmetric	60	33/60	35/60
5	10000	0.9	symmetric	60	60/60	60/60
5	10000	0.3	heterogeneous	60	39/60	37/60
5	10000	0.5	heterogeneous	60	29/60	29/60
5	10000	0.9	heterogeneous	60	41/60	41/60
8	10000	0.7	symmetric	40	37/40	37/40
8	10000	0.7	heterogeneous	40	6/40	9/40
10	10000	0.7	symmetric	30	18/30	18/30
10	10000	0.7	heterogeneous	30	4/30	4/30
20	10000	0.7	symmetric	30	6/30	7/30
20	10000	0.7	heterogeneous	30	1/30	1/30

In the standard procedure, the algorithm performs well for small d with symmetric marginals and strong dependence ($r_{\text{scale}} \geq 0.7$). Performance degrades with increasing d , heterogeneous marginals, and weak dependence. The degradation with heterogeneous marginals is particularly severe in higher dimensions, and recovery becomes essentially impossible.

A natural question is whether these failures are due to estimation error (i.e. noise in $\hat{\theta}_{ij}$) or to a structural limitation of the algorithm itself. Comparing the $d = 5$ rows at $n = 5000$ and $n = 10000$, the recovery rate is essentially unchanged, suggesting that the bottleneck is not estimation precision. Broadly, for $n \geq 1000$, changing the sample sizes did not affect results.

To develop this idea further, we tested an ‘oracle’ variant of the algorithm: at the first merge step, the algorithm is given the exact population θ matrix (from the true model) rather than the sample estimates. One might like to keep this ‘true’ θ matrix for the rest of the tree, but this is not possible, as the merging and recalculation steps of the algorithm require real data. For this reason, the oracle then switches to the standard sample-based computation for all subsequent steps. The results (above) show very minimal improvements for the oracle. In some cases the oracle actually performs slightly worse (e.g. $n = 5$, $r_{\text{scale}} = 0.3$, heterogeneous). Importantly, we can guess that the source of failure is not estimation noise but rather the greedy algorithm itself. Although this implementation is somewhat ad-hoc and not a rigorous analysis, it nonetheless shows that the algorithm as-is will not lead to success on real data.

4 Discussion

4.1 Implications

The simulation results from the previous section reveal that the greedy algorithms proposed above face some fundamental limitations. Given that we know that if the depth-ordered parametrization (Definition 16) were satisfied, the algorithm should completely recover the tree, we can theorize that this condition is being violated in some way. Particularly poor performance on trees with weak dependence or heterogeneous marginals indicates that these factors increase the degree to which the condition is violated. For example, heterogeneous marginals create more variation in the r_{S_k} values at different nodes, making it more likely that the θ ordering is disrupted. In such cases, even an algorithm with access to the exact θ matrix would select the wrong pair.

4.2 Directions for future work

4.2.1 Developments on the greedy algorithm

An interesting further investigation into the greedy paradigm would be to quantify how often the depth-ordered paradigm is satisfied across the random parameter draws in simulation, and to verify if the recovery rate corresponds to the satisfaction rate. We would conjecture that the two quantities are closely related.

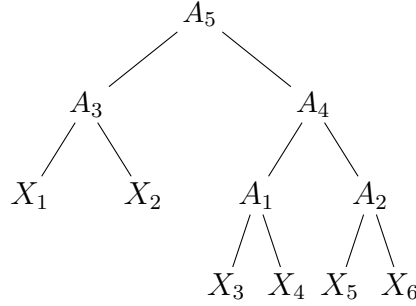
One could also explore how to enrich the oracle algorithm. Although this requires a true Θ matrix (and is therefore unsuitable for real data), being able to test algorithms on the entirely correct θ values is a useful way of evaluating their ability. Theorem 4 of from [VGN26] gives a closed-form expression for $\mathbb{P}[X_I = 1]$ for any subset $I \subseteq \{1, \dots, d\}$ as a product over first common ancestors in the tree. In particular, this could be leveraged to find a way to recalculate θ values for aggregated nodes after the first merge in the oracle approach.

4.2.2 Measuring partial recovery

The evaluation implemented in Section 3.2.2 is binary: either the algorithm recovers the exact unlabeled topology or it does not. While still in the algorithm development phase, a more informative metric could measure how near the learned tree is to the true one. It is possible that the greedy algorithm, for example, often recovers most of the tree but is just off by a few merges. Several notions of tree distance exist in the literature. The Robinson–Foulds distance [RF81], used in phylogenetics, counts the number of bipartitions (splits of the leaf set) that appear in one tree but not the other. One could also consider the tree edit distance, which measures the minimum number of elementary operations (node insertions, deletions, relabelings) needed to transform one tree into the other [ZS89].

A simpler metric which we can easily apply to the greedy algorithm is the fraction of correct merges: out of the $d - 1$ merge steps performed, how many produce pairs that are indeed siblings in the true tree?

Example 19. Consider a tree on $d = 6$ leaves where the true topology is



Suppose the algorithm produces the following merge sequence:

1. Merge X_1, X_2 into A_3 . (correct)
2. Merge X_5, X_6 into A_2 . (correct)
3. Merge X_3, X_4 into A_1 . (correct)
4. Merge A_3, A_1 into A'_4 . (incorrect, should merge A_1, A_2)
5. Merge A'_4, A_2 into A'_5 . (incorrect, from previous error)

Then 3 out of 5 merges are correct, giving a recovery fraction of $3/5 = 60\%$. This statistic provides more information than simply recording ‘failure’.

4.2.3 Sorting the Θ matrix

Another alternative approach would draw on the observation that, under the true tree, the Θ matrix has a block structure, per Proposition 15. Recovering the tree would then be made easier by finding a simultaneous permutation of the rows and columns of $\hat{\Theta}$ that reveals this block structure as clearly as possible. [PDN19] studied methods for sorting Kendall’s tau matrices to reveal hierarchical structure. Adapting such methods to the Θ matrix of the Bernoulli max-tree model could provide an alternative (potentially non-greedy) algorithm that exploits the global structure of the matrix instead of constructing a series of local decisions.

4.2.4 A more ‘delicate’ algorithm

Given the structural problems with the greedy approach, we should also consider algorithms which will be less sensitive to the ordering of the tree. One possibility is a *semi-greedy* approach, where instead of committing to the single best merge at each step, we could maintain a set of candidate merges (e.g. all pairs whose $\hat{\theta}_{ij}$ exceeds some threshold) and explore several merge

sequences, then select that which is the best fit (as measured by the Wald test). However, this method would be much more computationally expensive.

4.3 Applications

4.3.1 Financial data

Given a collection of d stocks, one could define binary indicator variables $X_i = \mathbf{1}_{\{|R_i| > c\}}$ where R_i is the return of stock i and c is a threshold. One then has an indicator of ‘extreme’ movement. Based on the heuristic that stocks within the same sector tend to move together, one would expect the learned tree to reflect sector structure. Similar indicators could be defined for a variety of price fluctuations in financial data.

One can also apply the model to credit risk, though related data often lacks the high number of dimensions that makes this model interesting. For example, in a portfolio of d obligors, let $X_i = \mathbf{1}_{\{\text{obligor } i \text{ defaults}\}}$. The max aggregation tree then models joint default, and the tree structure represents shared risk factors.

4.3.2 Weather extremes

One could also apply the model to extreme weather data from a network of monitoring stations. Given d stations, define $X_i = \mathbf{1}_{\{\text{temperature at station } i \text{ exceeds } c\}}$ for a threshold c . Given the autocorrelation present within spatial data, nearby stations should exhibit correlated extremes, such as during heat waves. This setting also provides an alternate, heuristic way to find the tree: one can construct a *geographic tree* by recursively merging the two geographically closest observation stations, producing a binary tree that reflects spatial proximity without reference to the data. The learned tree (say, from the greedy algorithm) could then be compared against this geographic tree using the distance metrics from Section 4.2.2, or simply tested for statistical significance via the Wald test in Section 3.3.1.

In any case, until we find an algorithm which can recover the tree all of the time (or at least most of the time, as validated by the Wald test), the applications of the work, will, unfortunately, continue to be limited.

References

- [AHM12] Philipp Arbenz, Christoph Hummel, and Georg Mainik. “Copula based hierarchical risk aggregation through sample reordering”. In: *Insurance: Mathematics & Economics* 51.1 (2012), pp. 122–133.
- [CG15] Marie-Pier Côté and Christian Genest. “A copula-based risk aggregation model”. In: *Canadian Journal of Statistics* 43.1 (2015), pp. 60–81.
- [DDW13] Bin Dai, Shuangge Ding, and Grace Wahba. “Multivariate Bernoulli distribution”. In: *Bernoulli* 19.4 (2013), pp. 1465–1483.
- [Eth37] I. M. H. Etherington. “Non-associate powers and a functional equation”. In: *Mathematical Gazette* 21 (1937), pp. 36–39, 153.
- [Nel06] Roger B. Nelsen. *An Introduction to Copulas*. 2nd ed. New York: Springer, 2006.
- [PDN19] S. Perreault, T. Duchesne, and J.G. Nešlehová. “Detection of block-exchangeable structure in large-scale correlation matrices”. In: *Journal of Multivariate Analysis*, 169 (2019), pp. 400–422.
- [RF81] D. F. Robinson and L. R. Foulds. “Comparison of phylogenetic trees”. In: *Mathematical Biosciences* 53.1–2 (1981), pp. 131–147.
- [Val26] Samuel Valiquette. “Tree structure and Wald test for aggregation Bernoulli”. Unpublished note, Department of Mathematics and Statistics, McGill University. 2026.
- [VGN26] Samuel Valiquette, Christian Genest, and Johanna G. Nešlehová. “A class of multivariate Bernoulli distributions generated by an aggregation-tree model”. Preprint, Department of Mathematics and Statistics, McGill University. 2026.
- [ZS89] Kaizhong Zhang and Dennis Shasha. “Simple fast algorithms for the editing distance between trees and related problems”. In: *SIAM Journal on Computing* 18.6 (1989), pp. 1245–1262.